#	Line(s)	Problem class	Why bug?	Manifested	Fix (minimal)
1	69-71	integer promotion	head, tail are uint8_t - applies usual arithmetic conversions §6.5.6p4 → §6.3.1.8 → integer promotions §6.3.1.1p2 int represents all uint8_t → both operands become int subtraction in int, not mod 256 → wraparound property lost	head=0, tail=248 (just after head wraps 255→0): (int)head - (int)tail = −248 → guard never fires (uint8_t)(head - tail) = 8 = correct occupancy head wraps after 256 pushes = 25.6 s @ 100 ms then: tail stranded ~248 behind, rb_pop() sees hundreds queued, returns overwritten slots	if ((uint8_t)(r->head - r->tail) > RB_SIZE) { r->tail = (uint8_t)(r->head - RB_SIZE); }
2	237	dead code	watchdog_kick() commented out surrounding timing	wdt expires at timeout if enabled in init periodic reset, looks like a freeze (" uređaj se povremeno zaledi")	uncomment the line

3	195-199	logic error, dead cod	Result of no_flags_set(low_bits, high_nibble_mask) is not used,
			if branch is empty
				cfg_is_healthy returns true for every cfg
				with correct SHTDN and ALERT bits despite
				other condition of having at least one bit in high nibble is set.
					if (no_flags_set(low_bits,
					high_nibble_mask)) {
					return false;
					}

4	135	unlimited blocking r	while (!i2c_start(...)) {} - no retry limit and no timeout	due to I2C bus errors, this could potentially
				block indefinitely
					uint8_t retries = 10;
					while (!i2c_start(MCP9808_ADDR, false))
					if (--retries == 0) return INT16_MIN;

5	141-144	unchecked error	i2c_start(MCP9808_ADDR, true) can return false, but the code does not check for that	This can manifest as a failed read and incorrect temperature data	add a retry loop or error handling for i2c_start() when reading
6	83-84	race condition, compiler optimization	Global variables modified in ISR are not declared volatile. The compiler may cache their values in registers, so the main loop never sees updates from the ISR when optimisations are enabled (-O2). This violates the requirement that objects shared with interrupt handlers must be volatile to prevent reordering and caching (C11 §5.1.2.3/4)	The main loop may hang forever waiting for sample_flag or send_flag; samples and transmissions are lost or delayed indefinitely.	Add volatile to both declarations: static volatile bool sample_flag = false; static volatile bool send_flag = false;
7	94-98	architectural and performance/timing bug	Cortex-M0+ has no FPU/hardware divide; float ops in ISR are slow.	Increased interrupt jitter; pulls soft-float library into the link.	Replace with integer: if (systick_count > 3600000u) { systick_count = 0; }

8	186	logic error	(left + right) / 2 assumes two halves of equal size; for BATCH_SIZE=5 the split is 2+3, so the smaller half gets the same weight as the bigger one, the result is not the arithmetic mean of 5 samples.
			Dont be smart, unnecessary flexing...
				BATCH_SIZE=5 → [10,20,30,40,50]
				gives 26 instead of true mean 30.
					Replace with iterative sum:
					int32_t sum = 0;
					for (...) sum += samples[i];
					return sum / n;

9					
10					
11					
12					
13					
14					
15					